

IUM Report

Introduction:

This report describes the data preparation and analysis conducted for the IUM project. Our work began with an inspection of the csv files to address basic issues such as NaN values, duplicates or useless rows in the datasets.

Based on our analysis goals, we performed a detailed cleaning of specific datasets like 'Oscars', 'Rotten Tomatoes', 'Genre' and 'Releases', along with some minor datasets such as 'Countries' and 'Movies'.

Finally, the project ends with a data visualization, where Seaborn and Plotly are used to show our final exploration.

Data Cleaning and Preparation:

The initial phase of the project focuses on the extraction, transformation, and cleaning of the datasets. Our primary focus was to ensure referential integrity and consistency

Solution:

For each dataset we cast columns to specific types to prevent logic errors during later analysis, we then applied a universal trim function using `str.strip()` to remove leading and trailing whitespaces, which can cause hidden duplicates or other problems.

We removed all the rows where essential information (like movie names in the movie dataset and links in the poster dataset) was missing, while keeping rows with non critical missing information (like the age rating in the releases dataset) to preserve as much information as possible.

For columns like `review_score` in the rotten tomatoes dataset and `age_rating` in the releases dataset, where we found messy values, we built custom logic (with regex and mapping dictionaries) to normalize diverse data into a single, comparable scale. By standardizing the scores of the rotten tomatoes reviews, we can directly compare these reviews with the ratings present in the movies database.

Issues:

A key part of our work involved cleaning up inconsistent formats, so that all the data is uniform and could be analyzed together:

- Chronological Misalignment: In the Oscars dataset, we discovered that ceremony years between 1928 and 1933 were shifted, making it look like the ceremonies happened before they actually did.

- **Format Ratings:** The review_score column in the Rotten Tomatoes dataset was a mix of letters (A+, C-), integers, and fractions with different denominators
- **Matching Identifiers:** The Oscar dataset and the Rotten Tomatoes one lacked the IDs found in the movies table, requiring a fuzzy matching logic to link them based on title of the film and release year.

Requirements:

This solution ensures our data preparation aligns with the project goals

- **Standardized Processing:** We used Python and the Pandas library to create a structured cleaning pipeline that ensures all datasets are processed consistently.
- **Schema Reliability:** Every CSV was transformed into a clean table format with corrected data types (strings, dates, and floats) to prevent errors during analysis.
- **Data Quality Control:** We actively improved the dataset's integrity by removing duplicates and strategically managing missing values to ensure reliable results.
- **Unified Logic:** By centralizing our transformation rules (like score normalization), we created a system that is both easy to maintain and adaptable to new data.

Limitations:

While the solution is robust, it has a few limitations:

- When we encountered scores like 3/5.4, 25/4 or 920, we had to decide whether to assume them as a typo or discard them. In those cases we chose to discard highly ambiguous data to prioritize accuracy over quantity.
- Our normalization for age ratings and scores relies on specific patterns. If new data with a completely new format (e.g., a "stars" emoji system) were added, the code would need to be updated.
- By labeling unknown age ratings as "Unrated," we lose some specific regional nuances from countries with non-standard labeling systems.

Database Design and Integration

Once the data was cleaned, we moved to extracting meaningful patterns, with a particular focus on how genres and awards influence movie performance.

Solution:

Design and its motivations we implemented a hybrid database architecture.

PostgreSQL was chosen for relational data to ensure referential integrity between movies, actors, and countries. MongoDB was used for the review dataset because its document-oriented structure better handles large volumes of hierarchical data without the overhead of complex SQL joins.

Issues:

The main challenge was handling two fundamentally different data types: structured movie metadata (actors, studios, awards) and large-scale, semi-structured review data. We also had to ensure the data fit within relational constraints, such as string length limits in SQL, and maintain consistent column naming across different datasets during the migration.

Requirements

The design meets all storage requirements:

- Relational Consistency: The "postgres_connection1.py" script standardizes names and slices Oscar categories to 254 characters to prevent VARCHAR overflow.
- Flexibility: MongoDB stores reviews in JSON format, allowing for high-speed ingestion and retrieval of critic evaluations.
- Automation: The pipeline automatically reads from the "/output" directory, ensuring that only the final, cleaned versions of the data reach the databases.

Limitations:

The "drop_existing=True" logic used in both scripts is destructive; it works for initial setup but isn't suitable for real-time data streaming where you'd need incremental updates. Additionally, the string slicing for SQL is a hard fix that might truncate valid data if award names exceed the character limit.

Data Analysis and Insights

Solution:

We adopted genre classification as our primary analytical dimension to investigate how thematic categories influence rating distributions. By aggregating our normalized scores, we calculated the average performance and popularity for each genre. We specifically investigated the distribution of ratings across different critic types, discovering that "Top Critics" maintain a much stricter scoring distribution compared to the general critic pool. We also identified some genres that consistently achieve higher scores over time. We analyzed the relationship between movie duration and ratings and if there were some patterns between the production of a certain genre and age_rating.

Issues:

There is a significant imbalance in the volume of data across genres (e.g., thousands of Dramas vs. hundreds of History films), which can skew average performance metrics if not properly weighted. Also, many movies are tagged with multiple genres. The analysis had to handle these overlaps to avoid double-counting or misrepresenting a film's primary thematic influence.

Requirements

We utilized dynamic visualizations to allow for multi-layered data exploration: We used

- Critic Insights: Confirmed that "Top Critics" are significantly more selective, with a tighter scoring distribution than standard critics.
- Oscar Correlation: Proved that Oscar winners show higher average ratings and lower variance, indicating a strong consensus between the Academy and professional reviewers.
- Production Trends: Identified a "sweet spot" for movie duration (120–150 minutes) where ratings tend to peak for narrative-heavy genres.

Limitations:

The analysis treats ratings as fixed points in time, failing to account for "re-evaluation" trends where older movies might gain or lose favor with modern critics. The dataset naturally favors movies that were successful enough to be archived and reviewed, potentially excluding thousands of "lost" or unrated independent films from the early 20th century.

Storytelling and Visualization

The final phase translated our findings into an interactive narrative, prioritizing clarity and user engagement over static charts.

Solution:

We utilized dynamic visualizations to allow for multi-layered data exploration, we developed "Genre Fingerprints" using Radar Charts, which allow for complete comparison of how different genres perform across multiple metrics like runtime, popularity, and average rating. To handle the complexity of time-based trends, we implemented interactive heatmaps; these provide a clean visual overview of genre evolution through color gradients, while still allowing the viewer to inspect precise data points via hover-tooltips. Furthermore, we utilized comparative subplots to place different genres, such as Drama and Comedy, side-by-side. This layout was essential to demonstrate a key insight: that even during production spikes where quantity increases, the average quality of these genres has remained remarkably consistent.

Issues:

A major challenge was representing the sheer volume of US-based movies without overshadowing international data. We solved this by using logarithmic scales in our choropleth maps and focused on "relative" performance metrics rather than just absolute counts.

Requirements

Our storytelling begins with blind data analysis, gradually connecting information - first in a broad, general way, then step by step moving toward a tighter, more specific analysis.

Limitations:

The most significant limitation is the 2020 Data Gap. We observed a sharp, anomalous drop in records starting from 2020. To avoid presenting misleading trends caused by incomplete data or the pandemic's impact on theatrical releases, we intentionally capped our temporal visualizations at 2019.

Conclusions

At the end of our analysis, we did not identify any particularly significant relationships between the production of certain movie genres over time and other factors such as age ratings, user ratings, Oscar nominations/wins, or film duration.

What proved most challenging was constructing a coherent narrative from the data. We found it particularly difficult to identify meaningful connections between the various datasets at our disposal. This difficulty was largely due to the sheer volume of data, which forced us to consider countless variables and exceptions. While we found this process

challenging, it was precisely this level of detail that allowed us to truly immerse ourselves in the data.

Working with real, tangible data transformed this project from a technical exercise into a concrete experience. The datasets were no longer just numbers and text—they represented real stories, actual films, and authentic reviews. This allowed us to mentally reconstruct the reality they described and connect emotionally with the material we were working on. As a result, we faced considerable challenges in filtering the data to work with consistent and useful information, but the richness of detail made the analysis feel genuinely connected to the real world of cinema.

Division of Work

We all had an equal part in the subdivision of the work. At the beginning we divided the 12 datasets, 4 for each of us and we all cleaned and normalized them. For the data analysis and the charts we had a draft about our story and then we tried to share them as equally as possible:

Cavallo developed the Oscars and Rotten Tomatoes datasets, involved correcting historical chronological shifts (1928–1933) and building a mapping system. Focused on the relationship between prestige (Oscars), critical reception, and global trends.

Andras focused on standardizing the Age Rating system across international releases, transforming fragmented labels into a consistent global categorization. Implemented the primary temporal analysis, using interactive visualizations to track how genre production and movie durations have evolved from 1970 to the present. Also studied the correlation between movies with specific genre-age_rating combo in time.

De Icco architected the core data infrastructure, meticulously cleaning and normalizing the actors, crew, countries, and genres datasets to ensure seamless cross-referencing.

Expanded the analytical scope into linguistic diversity, exploring the 'Language Barrier' by correlating the number of spoken languages within a film to its critical reception across different genres. Further bridged the gap between production and prestige by mapping global Oscar trends, synthesizing nomination and win data to visualize the international landscape of cinematic success."

Extra Information

We used some external libraries to normalize few string data

Check if you have some of these libraries on your device, if not you can add a cell at the beginning of the notebook, install the libraries and then you can remove the cell

```
# INSTALLATION CELL
```

```
%pip install plotly
```

```
%pip install fuzzywuzzy python-Levenshtein
```

```
%pip install statsmodels
```

```
%pip install textblob
```

```
%pip install transformers torch
```

```
%pip install num2words
```

```
%pip install recordlinkage
```

```
%pip install textdistance
```

The original cvs documents should be stored in a folder called "data" with the following path name "../solution/data"